Contents

Section	Title	Page
1	**Introduction**	**1**
1.1	Background of the Project	1
1.2	Problem Statement	2
1.3	Motivation	3
1.4	Objectives of the Project	4
1.5	Scope of the Project	5
1.5.1	In scope	5
1.5.2	Out of scope	5
1.6	Significance of the Project	6
1.7	Project Organization	6
1.7.1	Institutional context	7
1.7.2	Method of work	7
1.7.3	Document conventions	8
2	**Literature Survey**	**9**
2.1	Introduction	9
2.2	Existing Systems	9
2.2.1	General-purpose chat products	9
2.2.2	Prompt marketplaces and libraries	10
2.2.3	Image- and video-centric studios	10
2.2.4	Developer consoles	10
2.2.5	Low-code “AI wrappers”	10
2.3	Review of Related Works	11
2.3.1	Prompting as a research topic	11
2.3.2	Foundation-model risk	11
2.3.3	Software engineering and data	11
2.3.4	Human–computer interaction	11
2.3.5	Web platform literature	12
2.4	Limitations of Existing Systems	12
2.5	Research Gap	13
2.5.1	Gap analysis by stakeholder	13
2.6	Proposed Approach	14
2.7	Extended notes on prompting practice	14
2.8	Extended comparison remarks	15
2.9	Standards mapping	15
3	**System Analysis**	**16**
3.1	Existing System	16

Section	Title	Page
3.2	Proposed System	16
3.2.1	Actors	17
3.3	Functional Requirements	17
3.4	Non-Functional Requirements	18
3.5	Hardware Requirements	19
3.5.1	Development and demonstration	19
3.5.2	End-user device	19
3.6	Software Requirements	20
3.7	Feasibility Study	20
3.7.1	Technical feasibility	20
3.7.2	Economic feasibility	20
3.7.3	Operational feasibility	21
3.7.4	Schedule feasibility	21
3.7.5	Legal and ethical feasibility	21
3.7.6	Conclusion of analysis	21
4	**System Design**	**22**
4.1	System Architecture	22
4.2	System Workflow	23
4.3	Use Case Diagram	24
4.4	Data Flow Diagram	24
4.4.1	Context diagram (Level 0)	24
4.4.2	Level 1 processes	25
4.5	Entity Relationship Diagram	25
4.6	Database Design	26
4.6.1	Table: profiles	26
4.6.2	Table: prompts	26
4.6.3	Table: user_api_keys	26
4.6.4	Integrity and security	26
4.7	Module Design	27
4.8	User Interface Design	27
4.8.1	Visual language	27
4.8.2	Layout rules	27
4.8.3	Key screens	28
4.9	Data dictionary	28
4.10	Sample generated artefacts	28
4.10.1	Sample text prompt	28
4.10.2	Sample image prompt	29
4.10.3	Sample usage JSON	29

Section	Title	Page
4.11	User interface specification notes	29
4.12	Configuration checklist for the laboratory technician	30
4.13	Ethical use statement	30
5	**Methodology**	**31**
5.1	Proposed Methodology	31
5.1.1	Guiding principles	31
5.2	Module 1 — Authentication and Profile	31
5.2.1	Cloud path	32
5.2.2	Local path	32
5.2.3	Avatar	32
5.3	Module 2 — Prompt Studio (Generator and Library)	32
5.3.1	Controlled vocabulary	32
5.3.2	Composition	32
5.3.3	Persistence	33
5.3.4	Examples	33
5.4	Module 3 — Multi-Model Chat and Skills	33
5.4.1	Providers	33
5.4.2	Fan-out	33
5.4.3	Skills	33
5.4.4	Modality filter	33
5.5	Module 4 — Insights, Settings and Export	34
5.5.1	Usage	34
5.5.2	Settings	34
5.5.3	Export	34
5.6	Algorithms / Techniques Used	34
5.6.1	Algorithm A — Structured prompt composition	34
5.6.2	Algorithm B — Chat-compatible admission	34
5.6.3	Algorithm C — Parallel multi-model dispatch	35
5.6.4	Algorithm D — Usage aggregation	35
5.6.5	Techniques	35
5.7	Security Measures	35
6	**Implementation**	**37**
6.1	Development Environment	37
6.2	Front-End Implementation	38
6.2.1	Bootstrap	38
6.2.2	Application shell	38
6.2.3	Pages	38
6.2.4	Shared controls	38

Section	Title	Page
6.2.5	Responsive behaviour	39
6.3	Back-End Implementation	39
6.4	Database Implementation	39
6.5	API Integration	40
6.5.1	OpenRouter	40
6.5.2	Gemini	40
6.5.3	Failure handling	40
6.6	Authentication and Authorization	40
6.7	Module Implementation	41
6.7.1	Generator	41
6.7.2	AI Chat	41
6.7.3	Library	41
6.7.4	Dashboard and Overview	41
6.7.5	Settings and Profile	41
6.8	Security Implementation	42
6.9	Walk-through of a generate-and-save transaction	42
6.10	Walk-through of a chat send	43
6.11	Environment variables	43
6.12	Build and quality commands	43
7	**Testing**	**44**
7.1	Testing Strategy	44
7.2	Unit Testing	44
7.3	Integration Testing	45
7.4	System Testing	45
7.5	Security Testing	46
7.6	Performance Testing	46
7.7	Test Cases and Results	47
8	**Results and Discussion**	**49**
8.1	Experimental Setup	49
8.2	Implementation Results	49
8.3	Output Screenshots	50
8.4	Performance Analysis	51
8.5	Result Analysis	52
8.6	Discussion	52
9	**Conclusion and Future Scope**	**53**
9.1	Conclusion	53
9.2	Contributions of the Project	53

Section	Title	Page
9.3	Limitations	54
9.4	Future Enhancements	54
9.5	Future Scope	54
	References	**56**
	Books	56
	Research Papers	56
	Websites	57
	Documentation	57
	Standards / Technical References	57
B.1	**Placeholder frames**	**58**
	Bibliography	**61**

CHAPTER 1

Introduction

Artificial intelligence systems that generate text, images and short video descriptions have moved from research laboratories into classrooms, homes and small studios. A student who once struggled to start an essay can now request a draft. A teacher can ask for a worksheet. A designer can describe a poster and receive a prompt ready for an image model. This convenience hides a persistent difficulty: the machine does only what the instruction allows it to do. Vague instructions produce vague results. Unsafe instructions can produce unsuitable results. Vendor-specific chat boxes train users to talk to one brand, not to think about structure.

Promptora was conceived as an answer to that difficulty. It is a web application that treats the prompt itself as a first-class artefact. The user writes an idea in ordinary language, chooses a content type (text, image or video), sets audience, style, tone and length, and receives a structured prompt that can be copied, enhanced or saved. A second workspace, AI Chat, sends the same idea—or any follow-up question—to one or more chat-capable models through OpenRouter and Gemini. Keys belong to the user. Data belonging to an account is isolated by Supabase Row Level Security. The product is therefore both a learning tool and a small multi-model studio.

This chapter states the background, the problem, the motivation, the objectives, the scope, the significance and the organisation of the remainder of the report.

1.1 Background of the Project

The transformer architecture described by Vaswani et al. changed sequence modelling by replacing recurrence with attention [8]. Subsequent large language models demonstrated that a single pretrained network can perform many tasks when conditioned by a natural-language prompt [9, 10]. Prompting is therefore not a minor user-interface trick; it is the principal way that non-specialists program foundation models [11, 13].

Three background facts shaped Promptora.

Fact 1 — Prompt quality is uneven. Few-shot and instruction-tuned models still reward specificity: audience, constraints, format, safety and style. Beginners rarely supply these fields unless a form asks for them. White et al. catalogue reusable prompt patterns, which supports the idea that structure can be taught through a product, not only through a textbook [12].

Fact 2 — The model market is fragmented. OpenAI, Anthropic, Google, Meta and many open-weight providers expose different identifiers, rate limits and modalities. A student who wants to compare two chat models must open two websites and lose history. OpenRouter aggregates many chat models behind one HTTP interface; Gemini provides a native generateContent API. A campus product can sit in front of both if it stores the user's own keys and filters incompatible modalities.

Fact 3 — Educational settings need safety and ownership. Foundation models raise social and safety questions [14, 15]. A college project that invites school-age users must default to friendly wording, refuse clearly unsuitable model types in chat, and keep each learner's library private. Cloud persistence must not leak one student's prompts to another. Local fallback is useful in laboratories where a Supabase project has not yet been configured.

Against this background, a single-page React application with a prompt generator, a skill-aware multi-model chat, a library, usage charts and account pages is a realistic and useful undergraduate project. It applies software engineering [1, 2], relational design [3] and REST-style integration [7, 24] to a contemporary AI workflow.

1.2 Problem Statement

The problem addressed by this project can be stated as follows.

> *How can students, teachers and beginner creators convert a short natural-language idea into a clear, reusable, age-appropriate prompt for text, image and video models, and how can they compare multiple chat models from one authenticated workspace without surrendering ownership of API keys or personal prompt data?*

The problem decomposes into several practical failures of the status quo:

1. Users type one-line requests into chat products and receive inconsistent output because audience, tone and constraints were never stated.

2. Image and video prompting requires a different vocabulary (lighting, composition, camera, pacing) that chat interfaces do not systematically collect.

3. Switching models means switching accounts, losing a shared transcript and mixing incompatible endpoints (for example, sending a chat payload to a TTS or rerank model).

4. Hosted student projects often embed a single shared API key in environment variables, which is unsafe and unfair: one noisy user can exhaust the quota of the whole class.

5. Saved prompts, if they exist, are trapped inside a vendor product and cannot be exported as JSON, CSV, text or a print-ready PDF for a portfolio or a laboratory record.

6. Campus laboratories sometimes cannot reach a configured backend; the application then becomes a blank login wall instead of a usable tool.

Promptora solves these failures with a structured generator, chat-only model filtering, per-user keys, a cloud-or-local library, usage analytics and export.

1.3 Motivation

The team observed, in classroom demonstrations and in personal use of public chat tools, that the hardest step is not "calling the model" but "saying what you want." Students who can describe a story in Tamil or English still produce weak English prompts because they do not know which attributes matter. Teachers who want a lesson plan waste time rewriting the same constraints.

A second motivation is pedagogical. Building Promptora forces the team to prac-tise the complete software process: requirements, architecture, database schema with RLS, authentication, third-party APIs, responsive UI, testing and documentation [1]. The product is not a toy form; it is a small information system with real security boundaries.

A third motivation is fairness. By requiring each user to paste keys into Settings, the project avoids concentrating cost and risk on a single college-owned secret. Users who do not wish to chat can still generate and save prompts.

A fourth motivation is regional relevance. Shanmuga Industries Arts and Science College serves learners who need tools that are simple, inexpensive and safe. Promptora is designed so that a first-year student can generate a prompt in under a

minute, while a final-year student can inspect usage graphs and exported records.

1.4 Objectives of the Project

The primary objective is to design, implement, test and document Promptora as a complete web platform. The specific objectives are listed below.

1. To study prompting methods and existing multi-model products and to identify a research and product gap suitable for an undergraduate implementation [11, 12].
2. To specify functional and non-functional requirements for generator, chat, library, dashboard, profile, settings and public marketing pages.
3. To design a system architecture that separates presentation, client services, Supabase backend and external AI providers.
4. To implement a structured prompt builder that maps idea, model type, audience, style, tone, length and extras into a consistent instruction, with an optional enhancement pass.
5. To implement multi-model chat that accepts prompts from OpenRouter and Gemini, applies skill instructions, and rejects non-chat model identifiers.
6. To implement authentication with Supabase and a local fallback, including profile creation, avatar storage and protected routes.
7. To persist prompts and API-key metadata per user with PostgreSQL Row Level Security.
8. To track usage events and present graphs and export files on Overview and Dashboard.
9. To provide a responsive interface: top navigation on marketing pages, a dual sidebar on application pages, and a mobile bottom bar.
10. To evaluate the system through unit, integration, system, security and performance testing and to report results.

1.5 Scope of the Project

1.5.1 In scope

The following items are included in the delivered system and in this report:

- Public pages: Home, Features, About and Authentication.
- Application pages: Overview, Generator, AI Chat, Library, Dashboard, Profile and Settings.
- Prompt generation for three content types: text/story, image and video/storyboard.
- Multi-model chat with built-in and imported skills (manual, CSV, JSON).
- Cloud schema for profiles, prompts, user_api_keys and avatar storage.
- Local library and local auth when Supabase environment variables are absent.
- Usage event store, charts and CSV/JSON export; profile export as PDF (print-to-PDF), TXT and JSON.
- Responsive desktop and mobile layouts, copy-to-clipboard controls and friendly error messages.

1.5.2 Out of scope

The following items are intentionally excluded:

- Hosting a college-owned LLM or fine-tuning a model on campus hardware.
- Generating binary images or video files inside the browser; Promptora produces prompts and chat text, not media files.
- Payment, subscription billing or marketplace of prompts.
- Native Android or iOS applications (the web app is mobile-responsive).
- Administrative dashboards for college staff beyond what a normal authenticated user can see of their own data.
- Real-time collaborative editing of a single prompt by many users.

1.6 Significance of the Project

The significance of Promptora can be viewed from four angles.

Academic significance. The project is a concrete case study in modern full-stack engineering: component-based UI, context-driven auth, service modules, SQL with RLS, and HTTPS APIs. It can be used in software engineering and web-technology laboratories as a reference implementation.

Educational significance. By exposing fields such as audience and tone, the interface teaches prompt hygiene. Skills act as reusable system prompts, which is the same idea as a style guide. Students learn that "talking to AI" is closer to specification writing than to casual chat.

Social significance. Age-friendly defaults and the refusal of non-chat models reduce accidental misuse. Per-user keys and RLS reduce the chance that one account reads another account's library.

Professional significance. The team practised versioned front-end builds, environment configuration, schema migration via SQL files, and report writing—skills expected in entry-level software roles.

1.7 Project Organization

The remainder of this report is organised as follows.

- **Chapter 2 — Literature Survey** reviews prompting research, existing products and the gap that Promptora fills.
- **Chapter 3 — System Analysis** compares the existing and proposed systems and records requirements and feasibility.
- **Chapter 4 — System Design** presents architecture, workflow, use case, DFD, ER model, database and interface design.
- **Chapter 5 — Methodology** describes the four major modules and the algorithms and security measures used.
- **Chapter 6 — Implementation** explains the development environment and the realisation of front end, back end, database, APIs and security.
- **Chapter 7 — Testing** documents strategy, test levels, cases and results.

- **Chapter 8 — Results and Discussion** interprets screens, performance and outcomes.
- **Chapter 9 — Conclusion and Future Scope** summarises contributions and future work.

Table 1.1 records the project team as registered with the College.

Table 1.1: Project team members, Shanmuga Industries Arts and Science College

S. No.	Name	Register No.	Primary contribution
1	Tamilsevan D	20924U18177	Architecture, generator core, report coordination
2	Tharunkumar D	20924U18178	Authentication, Supabase schema, profile
3	Tharun R	20924U18179	AI Chat, model filter, API integration
4	Theerthagiri M	20924U18180	Library, usage analytics, dashboard
5	Thirumaran V	20924U18181	UI/UX, responsive layout, settings

The team worked as a single delivery unit. The contribution column indicates leadership of an area, not exclusive ownership. Design reviews, integration and testing were shared. This organisation matches recommended practice for small software teams [2].

1.7.1 Institutional context

The work was carried out in the Department of Computer Science, Shanmuga Industries Arts and Science College, Tiruvannamalai, affiliated to Thiruvalluvar University, during the academic year 2025–2026. Laboratory machines running Microsoft Windows, a browser (Chromium-based) and Node.js were sufficient to run the Vite development server. The cloud backend, when used, was a free-tier Supabase project. External model calls were made only when a team member supplied a personal OpenRouter or Gemini key.

1.7.2 Method of work

The team followed an incremental, prototype-driven process rather than a single

waterfall pass. An early generator page proved the prompt-building idea. Authentication and library persistence were added next. Multi-model chat, settings, usage and responsive navigation arrived in later increments. Each increment was demonstrated internally before the next feature was started. This is consistent with evolutionary development described by Sommerville [1].

1.7.3 Document conventions

In this report, monospaced text denotes file names, table names, routes and code identifiers. The product name Promptora refers to the implemented system. “User” means an authenticated or guest human. “Model” means a remote generative model identified by a provider-specific string. Figures that are drawn with TikZ are schematic; screenshots in Chapter 8 and Appendix B represent the running application.

The introduction has defined why the project exists and what it sets out to achieve. The next chapter examines published work and competing systems in more detail so that the design decisions in later chapters can be justified.

CHAPTER 2

Literature Survey

2.1 Introduction

A literature survey for an applied software project has two jobs. It must show that the team understands the scientific and industrial context, and it must locate a gap that a student implementation can honestly fill. This chapter therefore reviews (i) prompting and foundation-model research, (ii) existing commercial and open tools, (iii) related academic and student systems, (iv) limitations of those systems, (v) the research and product gap, and (vi) the approach adopted in Promptora.

The survey is not a claim that Promptora invents large language models. The contribution is a carefully scoped platform: structured prompting, multi-model chat with modality filters, user-owned keys, and privacy-aware persistence [11, 1].

2.2 Existing Systems

2.2.1 General-purpose chat products

ChatGPT, Gemini web, Claude.ai and similar products provide a conversational box, optional file upload and a single-vendor model switcher. They are excellent for casual use. Their limitations for the problem stated in Chapter 1 are structural:

- The prompt is ephemeral. There is no first-class library of reusable, typed prompts (text / image / video) with audience metadata.
- The user is trained to chat, not to fill a specification. Constraints must be remembered and retyped.
- Comparison across vendors requires multiple subscriptions and multiple histories.
- Export of a personal corpus as JSON or CSV is limited or absent.
- Educational defaults (child-safe wording, classroom audience) are not the primary design centre.

2.2.2 Prompt marketplaces and libraries

Websites such as PromptBase, FlowGPT and various “prompt of the day” blogs sell or share prompt text. They treat prompts as content, not as an interactive builder. A student still has to adapt a purchased prompt by hand. There is usually no per-user cloud schema with RLS, no multi-model execution from the same page, and no usage graph tied to a college account.

2.2.3 Image- and video-centric studios

Midjourney (Discord), Leonardo, Runway and similar tools specialise in a modality. They collect some visual parameters but do not unify text-story, image and video-storyboard prompting, nor do they sit in front of arbitrary OpenRouter chat models. They also assume a paid studio workflow rather than a campus laboratory.

2.2.4 Developer consoles

OpenAI Playground, Google AI Studio and OpenRouter’s own chat UI are powerful but assume that the user already understands model identifiers, system prompts and temperature. They are not designed for school students. They do not save a pedagogical library of generated prompts with audience and tone fields.

2.2.5 Low-code “AI wrappers”

Many student and startup projects wrap a single /chat/completions call around a textarea. These wrappers typically:

- store one API key in .env on a server;
- offer no model-type validation;
- omit library, dashboard and export;
- collapse on mobile;
- provide no local fallback if the backend is missing.

Promptora is deliberately broader than a wrapper and narrower than a full creative suite.

2.3 Review of Related Works

2.3.1 Prompting as a research topic

Liu et al. survey prompting methods across NLP and organise them into a "pre-train, prompt and predict" paradigm [11]. Reynolds and McDonell argue that prompt programming goes beyond few-shot exemplars and should be treated as a design activity [13]. White et al. propose a catalog of prompt patterns analogous to software design patterns [12, 5]. These works justify a product that makes pattern fields visible: audience, style, tone, length and extras.

Instruction following improved after reinforcement learning from human feedback [10]. That improvement does not remove the need for structure; it makes structured prompts even more effective because the model is trained to obey them.

2.3.2 Foundation-model risk

Bommasani et al. and Weidinger et al. discuss opportunities and harms [14, 15]. For a college project the practical response is not a full alignment laboratory; it is product-level mitigation: safe default copy, refusal of unsuitable model classes in chat, user-owned keys, and private rows in Postgres.

2.3.3 Software engineering and data

Classical software engineering texts remain the backbone of the process model used here [1, 2]. Database textbooks inform normalisation, keys and integrity [3]. REST and HTTP semantics govern how the browser talks to Supabase and to model providers [7, 24, 26]. JWT-based sessions are the industry standard for SPA authentication [25]. Quality characteristics follow ISO/IEC 25010 thinking (usability, security, reliability, portability) [27]. Security review is guided by the OWASP Top 10, especially broken access control, injection and security misconfiguration [28].

2.3.4 Human–computer interaction

Nielsen's usability heuristics—visibility of system status, match with the real world, user control, consistency, error prevention, recognition rather than recall—directly influenced the live preview, copy confirmation, friendly auth errors and segmented content-type control [6]. A dark, high-contrast studio theme reduces glare in computer laboratories.

2.3.5 Web platform literature

Official documentation for React, Vite, React Router, Supabase, OpenRouter and Gemini defined the implementation surface [16, 17, 18, 19, 20, 21]. PostgreSQL documentation guided RLS policy writing [23].

2.4 Limitations of Existing Systems

Table 2.1 summarises the comparison used during analysis.

Table 2.1: Comparison of existing approaches and Promptora

Feature	**Vendor chat**	**Prompt shop**	**API play-ground**	**Promptora**
Structured fields	Weak	None	Manual	First-class
Text / image / video prompts	Partial	Mixed	Manual	Unified generator
Multi-vendor chat	Rare	No	Partial	OpenRouter + Gemini
Chat-only filter	Vendor-specific	No	User must know	Enforced
User-owned keys	N/A	N/A	Yes	Yes, per account
Private library	Limited	No	No	RLS + local
Usage graphs	Vendor analytics	No	Quotas	In-app events
Campus fallback	No	No	No	Local auth/library
Age-safe defaults	Variable	No	No	Built into builder
Export portfolio	Weak	No	No	PDF / TXT / JSON / CSV

The limitations that matter most for this project are:

1. **Lack of a specification-oriented UI.** Chat hides the attributes that make prompts reliable.
2. **Vendor lock-in.** Histories and styles do not travel.

3. **Unsafe shared secrets** in classroom wrappers.
4. **No modality guard** when a user pastes an arbitrary model id.
5. **Weak educational export** for viva and record notebooks.
6. **No graceful degradation** without a configured backend.

2.5 Research Gap

From the review, the gap can be phrased as:

> *There is no lightweight, education-centred, open-implementation web platform that (a) builds structured multi-modal prompts, (b) chats with several providers using the learner's own keys, (c) filters non-chat models,*
> *(d) stores artefacts under RLS, and (e) degrades to local storage when cloud configuration is missing.*

Filling this gap does not require a new training algorithm. It requires product research, careful requirements, security-aware data design and a disciplined React implementation. That is an appropriate gap for a B.Sc. Computer Science project: scientifically informed, technically non-trivial, and completable in one academic year.

2.5.1 Gap analysis by stakeholder

Student. Needs a form that asks the right questions and a library that survives the laboratory session.

Teacher. Needs safe defaults and exportable examples for class.

Developer-student (the team). Needs a system that exercises auth, SQL, APIs, routing and testing.

Institution. Needs a project that can be demonstrated offline-ish (local fallback) and online (Supabase).

No reviewed system served all four stakeholders at once.

2.6 Proposed Approach

Promptora's approach is summarised as **specify – generate – converse – store – reflect**.

1. **Specify.** Collect idea and metadata through a dedicated generator form.
2. **Generate.** Compose a deterministic prompt using maps for audience, style, tone and length; optionally enhance.
3. **Converse.** Send messages to one or more chat-compatible models with optional skills.
4. **Store.** Save prompts to Supabase or localStorage; save keys only in the user's row or local cache.
5. **Reflect.** Record usage events and show graphs; export for records.

Architectural principles adopted from the literature include separation of concerns [5], client–server resource APIs [7], and defence in depth for access control [28]. Usability principles include recognition over recall and visible status [6].

The next chapter converts this approach into a systematic analysis of existing versus proposed systems and into testable requirements.

2.7 Extended notes on prompting practice

Classroom observation, even without a formal experiment, shows three recurring student behaviours.

First, students under-specify the audience. They write "explain photosynthesis" and receive a university-level paragraph that a sixth-standard learner cannot use. Promptora's audience field forces a choice.

Second, students mix modalities. They ask a chat model to "make a poster" and are disappointed when they receive paragraphs instead of an image prompt. Separating text, image and video in the generator, and blocking non-chat models in AI Chat, reduces that confusion.

Third, students lose good prompts. A sentence that worked last week is gone because the vendor chat was cleared. A library with copy and export treats the prompt as homework, not as chat residue.

These observations are consistent with the literature that treats prompting as design [13, 12] and with Nielsen's rule that systems should reduce memory load [6].

2.8 Extended comparison remarks

Vendor chat products will remain more powerful than Promptora at open-ended reasoning. That is not a failure of this project. Promptora's job is orchestration and education. A calculator does not compete with a mathematician; it makes arithmetic inspectable. Similarly, Promptora makes the instruction inspectable.

Prompt shops commoditise text but do not help a Tamil-medium student invent a first prompt. Playgrounds assume fluency in model identifiers. Wrappers hide keys on a server and fail in a locked-down lab. The combination implemented here is the contribution.

2.9 Standards mapping

Table 2.2 maps product features to the standards cited in this chapter.

Table 2.2: Standards and Promptora features

Standard	**How it appears in Promptora**
ISO/IEC 25010 usability	Preview, examples, friendly errors
ISO/IEC 25010 security	RLS, route guards, user keys
ISO/IEC 25010 portability	Static Vite build
RFC 9110	HTTPS JSON APIs
RFC 8259	JSON exports and jsonb models
RFC 7519	Supabase session JWT
OWASP A01	RLS and storage path checks
OWASP A02	No shared model secret in git

The mapping is qualitative. No certification audit was purchased. It exists to show the team read the standards instead of only copying UI from a tutorial.

CHAPTER 3

System Analysis

System analysis translates the problem and the literature gap into a precise description of what will be built. This chapter describes the existing informal system, the proposed Promptora system, functional and non-functional requirements, hardware and software requirements, and feasibility.

3.1 Existing System

Before Promptora, the "system" used by the target audience was a patchwork:

- A browser tab open on a public chatbot.
- Notes in a phone or a Word file containing half-remembered prompts.
- Occasional copy-paste into an image website.
- No common account, no typed metadata, no usage history that the learner owns.

In computer-science laboratories the existing system is even weaker: a shared machine, a shared bookmark, and sometimes a shared API key written on the blackboard. Failures include lost work at logout, leakage of keys, inability to compare models, and no audit trail for a project viva.

Data flow in the existing system is linear and lossy: idea → chat box → output → forgotten. There is no store, no schema, and no access-control policy.

3.2 Proposed System

The proposed system is a Vite + React single-page application named Promptora. It offers:

- Marketing pages that explain the product without requiring login.
- An authentication page with sign-up (name, email, password) and sign-in.
- A generator that produces structured prompts.
- AI Chat with skills and multiple models.

- A library of saved prompts.
- Dashboard and Overview with statistics and charts.
- Profile and Settings, including API keys and custom models.

Cloud mode uses Supabase Auth, Postgres and Storage. Local mode uses localStorage for session, library and keys. Protected routes redirect guests to /auth.

3.2.1 Actors

1. **Guest** — browses Home, Features, About, Generator (without cloud save) and Auth.
2. **Registered user** — all guest abilities plus library, dashboard, profile, settings and cloud sync.
3. **External AI provider** — OpenRouter or Gemini, invoked only with the user's key.
4. **Supabase platform** — identity, database and storage.

3.3 Functional Requirements

Functional requirements (FR) describe services the system must provide [1].

ID	Requirement
FR-01	The system shall provide public pages Home, Features and About.
FR-02	The system shall allow a user to sign up with name, email and password.
FR-03	The system shall allow a user to sign in and sign out.
FR-04	The system shall create a profile row automatically after successful cloud sign-up.
FR-05	The system shall redirect unauthenticated users away from Library, Dashboard, Profile and Settings.
FR-06	The generator shall accept idea, content type, audience, style, tone, length and extras.
FR-07	The generator shall build a structured prompt and show a live preview.
ID	**Requirement**

FR-08	The generator shall allow an enhancement pass on an existing prompt.
FR-09	The generator shall allow copy to clipboard and save to library when permitted.
FR-10	AI Chat shall send a user message to every selected chat-compatible model.
FR-11	AI Chat shall reject video, TTS, rerank and image-only model identifiers.
FR-12	AI Chat shall support built-in, manual and imported skills.
FR-13	Settings shall store OpenRouter and Gemini keys per user.
FR-14	Settings shall allow custom model identifiers to be added.
FR-15	Library shall list, copy and delete the current user's prompts.
FR-16	Dashboard and Overview shall show usage counts and graphs.
FR-17	Overview shall export usage as CSV and JSON.
FR-18	Profile shall update name, bio and avatar and export a personal archive.
FR-19	The layout shall hide the app sidebar on marketing and auth pages.
FR-20	The layout shall present a mobile bottom navigation on small screens.

3.4 Non-Functional Requirements

Non-functional requirements (NFR) follow quality characteristics in ISO/IEC 25010 [27].

ID	Requirement
NFR-01	**Usability.** A new user shall generate a first prompt in under two minutes without a manual.
NFR-02	**Responsiveness.** Desktop (≥ 960 px) shall use the dual sidebar on app pages; smaller viewports shall use hamburger + bottom nav.
NFR-03	**Security.** Prompt and key rows shall be readable only by the owning user (RLS).
NFR-04	**Security.** Model API keys shall not be taken from the Vite environment for chat calls.
NFR-05	**Reliability.** If Supabase is unconfigured, local auth and local library shall still function.

NFR-06	**Performance.** Initial Vite production build shall load the interactive shell in a few seconds on laboratory broadband.
NFR-07	**Compatibility.** The application shall run on current Chromium and Firefox browsers.
NFR-08	**Maintainability.** UI, services, lib helpers and SQL shall live in separate folders.
NFR-09	**Portability.** The front end shall be a static build deployable to any static host.
NFR-10	**Error handling.** Auth and model errors shall be mapped to friendly English sentences.

3.5 Hardware Requirements

3.5.1 Development and demonstration

Table 3.3: Minimum hardware for development

Item	Specification
Processor	Dual-core 2.0 GHz or higher
Memory	8 GB RAM (16 GB recommended)
Storage	10 GB free for Node modules and build artefacts
Display	1366 × 768 minimum; 1920 × 1080 preferred
Network	Internet access for npm, Supabase and model APIs

3.5.2 End-user device

A smartphone with a modern browser is sufficient for marketing pages and generator use. Chat and dashboard are more comfortable on a tablet or laptop. No GPU is required on the client because generation happens on provider infrastructure.

3.6 Software Requirements

Table 3.4: Software stack

Layer	Software
OS (dev)	Windows 10/11 (team laboratory), also portable to macOS/Linux
Runtime	Node.js 20 or later, npm
Language	JavaScript (ES modules), JSX
UI library	React 19
Bundler	Vite 8
Routing	React Router 7
Backend BaaS	Supabase (Auth, Postgres, Storage)
Database	PostgreSQL 15+ (hosted)
AI APIs	OpenRouter Chat Completions, Gemini generate-Content
Versioning	Git (optional for submission archive)
Report	LaTeX (pdfLaTeX + biber)
Browser	Chrome / Edge / Firefox current

Environment variables required for cloud mode are only VITE_SUPABASE_URL and VITE_SUPABASE_ANON_KEY. Model keys are user-supplied at runtime.

3.7 Feasibility Study

A feasibility study asks whether the project should proceed [2]. Four dimensions were examined.

3.7.1 Technical feasibility

React, Vite and Supabase are mature, well documented and free at student scale [16, 17, 19]. OpenRouter and Gemini expose HTTPS JSON APIs that a browser can call with the user's key [20, 21]. RLS is a built-in PostgreSQL feature [23, 3]. The team already used JavaScript in coursework. Technical risk is moderate and manageable.

3.7.2 Economic feasibility

All core tools have a free tier. Domain and paid model usage, if any, are borne by the individual who creates a key. The college does not need to purchase GPU servers. Economic feasibility is high.

3.7.3 Operational feasibility

Staff can demonstrate the product in a browser. Students can use the generator without keys. Chat is optional. Operational feasibility is high for a campus setting.

3.7.4 Schedule feasibility

The academic calendar allowed incremental delivery: analysis, prototype, auth, chat, polish, test, report. Schedule feasibility is acceptable because the scope excluded media-file generation and billing.

3.7.5 Legal and ethical feasibility

The team uses only public APIs under their respective terms. Users are responsible for content they generate. The product defaults to safe wording. No personal dataset of minors is collected beyond account fields the user types. This is ethically acceptable for a college project.

3.7.6 Conclusion of analysis

The proposed system is feasible, necessary and well bounded. Chapter 4 turns the requirements into architecture and detailed design.

CHAPTER 4

System Design

Design converts requirements into a blueprint that implementers can follow [1, 2]. This chapter presents architecture, workflow, use case, data-flow, entity-relationship, database, module and user-interface design for Promptora.

4.1 System Architecture

Promptora uses a three-lane architecture:

1. **Presentation lane** — React pages and shared components (Navbar, App-Sidebar, MobileBottomNav, CopyButton, UsageChart).
2. **Application lane** — contexts (AuthContext) and services (promptStore, apiKeysStore, skillsStore, usageStore, profileStore, aiChat).
3. **Resource lane** — Supabase (Auth, Postgres, Storage), localStorage fallback, OpenRouter and Gemini HTTPS APIs.

The browser never embeds a privileged service-role key. Only the anon key is present in the Vite environment, and RLS enforces per-user isolation.

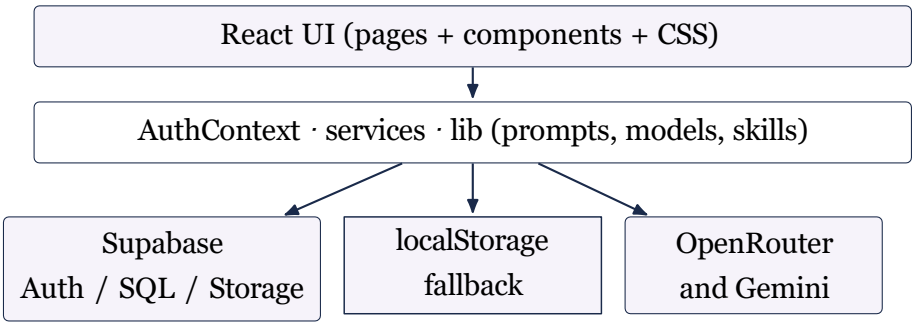


Figure 4.1: High-level architecture of Promptora

Routing is centralised in App.jsx. Marketing paths (/, /auth, /features, /about) omit the desktop sidebar. Application paths show the sidebar from 960 px upward. Footer is hidden on Auth and AI Chat to maximise working space.

4.2 System Workflow

A typical successful session proceeds as follows.

1. The user opens the Home page and optionally reads Features.
2. The user signs up or signs in. Supabase issues a session JWT [25]; a trigger inserts a profile.
3. The user opens Settings and pastes OpenRouter and/or Gemini keys.
4. The user opens Generator, fills the form, previews, generates, optionally enhances, copies or saves.
5. The user opens AI Chat, sets one or more chat models, chooses a skill, and sends a message. Replies appear in parallel cards.
6. The user reviews Library and Dashboard. Usage events recorded during generate/chat appear as charts.
7. The user exports a profile archive or usage CSV if required for a record notebook.

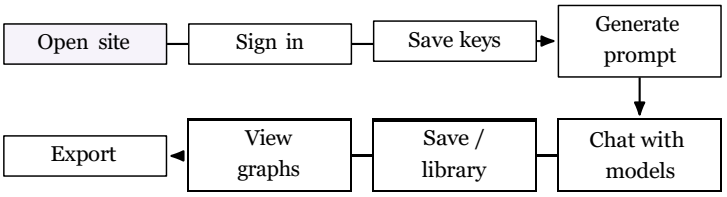


Figure 4.2: Primary user workflow

4.3 Use Case Diagram

Figure 4.3 shows the principal use cases. Guest use cases are a subset of registered-user use cases plus public browsing.

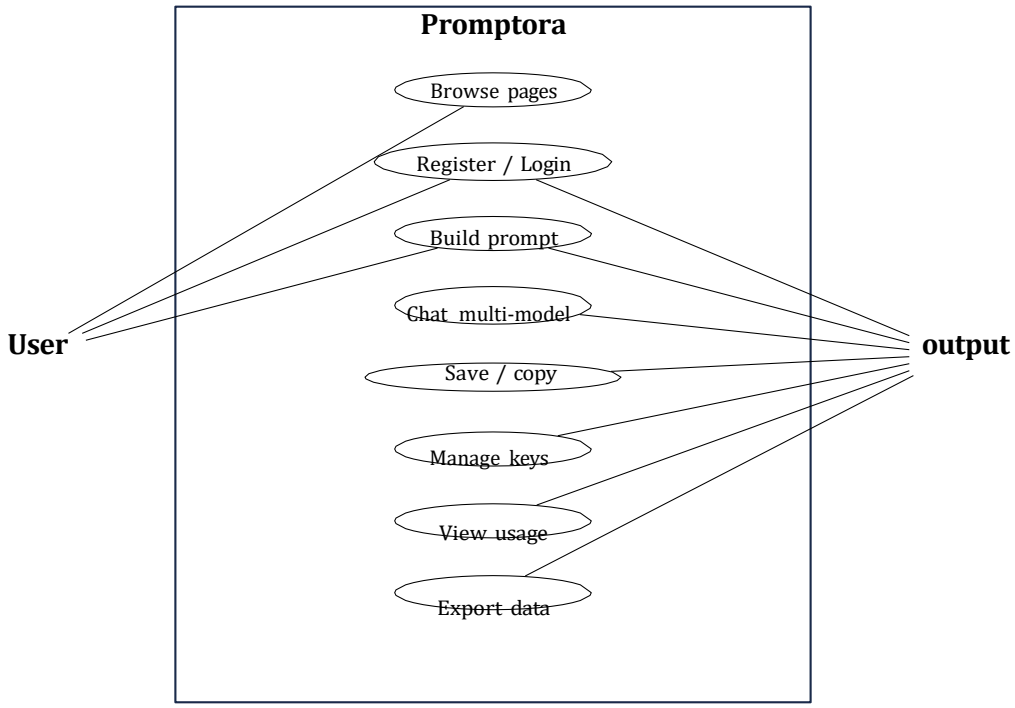


Figure 4.3: Use case diagram

4.4 Data Flow Diagram

4.4.1 Context diagram (Level 0)

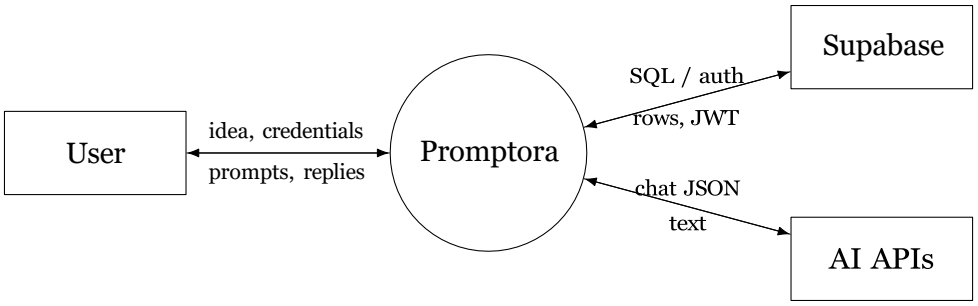


Figure 4.4: DFD Level 0 — context diagram

4.4.2 Level 1 processes

The Level 1 decomposition contains four processes:

1. P1 Authenticate and manage profile.
2. P2 Build, enhance and persist prompts.
3. P3 Chat with filtered models and skills.
4. P4 Aggregate usage and export.

Stores are D1 Profiles, D2 Prompts, D3 API keys, D4 Usage events (client), D5 Skills (client + optional custom).

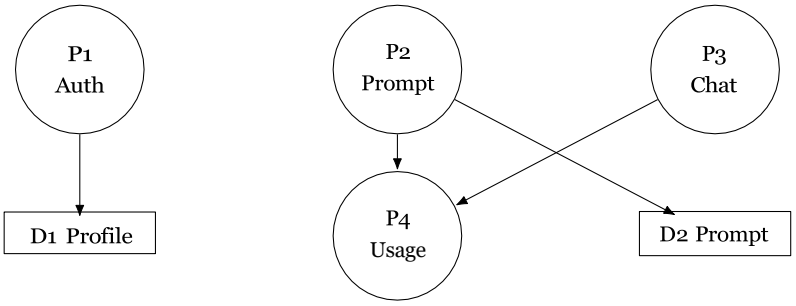


Figure 4.5: DFD Level 1 — process overview

4.5 Entity Relationship Diagram

The cloud conceptual model contains User (from auth.users), Profile, Prompt and UserApiKeys. Avatar objects live in Storage and are referenced by URL.

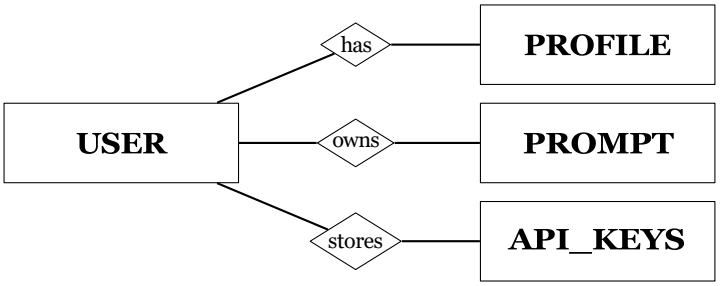


Figure 4.6: Entity relationship diagram (conceptual)

Cardinalities: one user has one profile (1:1), one user owns many prompts (1:N), one user stores one API-key row (1:1). Prompts do not exist without a user (identifying relationship via user_id).

4.6 Database Design

4.6.1 Table: profiles

Table 4.1: Relation profiles

Column	Type	Key	Notes
id	uuid	PK, FK	References auth.users(id)
email	text		Copied from auth
name	text		Display name
bio	text		Optional
avatar_url	text		Public storage URL
created_at	timestamptz		Default now()
updated_at	timestamptz		Maintained on upsert

4.6.2 Table: prompts

Table 4.2: Relation prompts

Column	Type	Key	Notes
id	uuid	PK	gen_random_uuid()
user_id	uuid	FK	Owner
title	text		Derived from idea
idea	text		Original idea
model	text		Check: text, image, video
audience, style, tone	text		Generator metadata
prompt	text		Generated instruction
created_at	timestamptz		Indexed with user_id

4.6.3 Table: user_api_keys

Table 4.3: Relation user_api_keys

Column	Type	Key	Notes
user_id	uuid	PK, FK	One row per user
openrouter_key	text		User-provided
gemini_key	text		User-provided
custom_models	jsonb		Default []
created_at, updated_at	timestamptz		Audit

4.6.4 Integrity and security

All three tables enable RLS. Policies restrict SELECT, INSERT, UPDATE and DELETE

to auth.uid() = id or auth.uid() = user_id. Profile insertion on sign-up is performed by a SECURITY DEFINER trigger so that RLS does not block the first row. Avatar upload paths must start with the user's UUID folder name.

These decisions follow standard multi-tenant relational design [3] and mitigate broken access control [28].

4.7 Module Design

The software is decomposed into the modules detailed in Chapter 5. Here the design-time responsibilities are fixed.

Table 4.4: Logical modules

Module	Responsibility
M1 Auth and profile	Session, local fallback, avatar, protected routes
M2 Prompt studio	Maps, buildPrompt, enhancePrompt, library I/O
M3 Multi-model chat	Provider calls, skill catalog, modality filter
M4 Insights and settings	Usage events, charts, export, key/model settings

Shared libraries (src/lib) stay free of React components so that prompt construction and model rules can be tested as pure functions.

4.8 User Interface Design

4.8.1 Visual language

The interface uses a near-black canvas (#000000), violet accents (#7c3aed, #a78bfa) and the Roobert / fallback sans-serif stack. Marketing pages use a full-width hero and a top navbar. Application pages use a 52 px icon rail plus a 240 px panel, collapsing to an icon rail only.

4.8.2 Layout rules

- Breakpoint 960 px: desktop sidebar visible; top navbar hidden on app pages.
- Below 960 px: hamburger menu and fixed bottom navigation.
- Auth page: centred card, navbar visible so the user can leave.
- Touch targets: minimum 44 px on small screens.

4.8.3 Key screens

Home presents rotating benefit words, trusted-brand strip, three-step explanation and FAQ. Generator is a two-column form and preview. AI Chat is a left control column and a message column. Library is a card grid. Dashboard shows identity, stats and graphs. Settings uses stacked cards for keys and custom models. Profile combines a form and export actions.

Wireframe-level structure is sufficient at design time because the implementation is CSS from the start rather than a separate mock tool. Nielsen's heuristic of consistency is enforced by shared buttons, eyebrows and page shells [6].

The design is now specific enough to implement. Additional dictionary, samples and laboratory notes follow; Chapter 5 then records the methodology and algorithms used inside each module.

4.9 Data dictionary

A data dictionary records every persistent field so that future maintainers do not guess meanings [3].

Field	Store	Meaning
profiles.id	Postgres	Same as auth user UUID
profiles.name	Postgres	Public display name
profiles.bio	Postgres	Short self-description
profiles.avatar_url	Postgres	HTTPS URL or empty
prompts.idea	Postgres	Original user sentence
prompts.prompt	Postgres	Generated instruction
prompts.model	Postgres	text / image / video
user_api_keys.openrouter_key	Postgres	Bearer token for OpenRouter
user_api_keys.gemini_key	Postgres	Gemini API key
user_api_keys.custom_models	Postgres	JSON array of {id,label,…}
promptora_library	localStorage	Guest/local prompt array
promptora_usage_events	localStorage	Event log, max 2000
promptora_sidebar_collapsed	localStorage	"1" or "0"

4.10 Sample generated artefacts

4.10.1 Sample text prompt

Write high-quality text content based on this idea: "A robot learning to paint with kids". Audience: students

learning a topic. Tone: warm and approachable. Include rich detail, structure, constraints, and concrete examples. Aim for a balanced medium length. Structure: clear opening, organized body, and a useful ending. Prefer plain readable text. Avoid heavy markdown symbols. Keep the result safe, friendly, and suitable for all ages.

4.10.2 Sample image prompt

Create a detailed image-generation prompt based on this idea: "Floating night library". Audience: a general audience of all ages. Tone: warm and approachable. Include subject, setting, lighting, mood, color palette, composition, and art style. Avoid logos, watermarks, unsafe content, or real-person likeness requests. Keep the result safe, friendly, and suitable for all ages.

4.10.3 Sample usage JSON

```
{
  "generatedAt": "2026-09-13T10:00:00.000Z",
  "rangeDays": 30,
  "totals": {
    "generator_generate": 12,
    "chat_send": 5,
    "generator_save": 4
  }
}
```

Listing 4.1: Usage export fragment

4.11 User interface specification notes

Every primary button uses the shared .btn class. Destructive actions (delete) remain ghost/outlined to avoid accidental clicks. Empty states always offer a link to the generator. Status messages use green for success and red for error, never colour alone without text (accessibility).

Form labels sit above controls. Segmented type selectors show an icon plus a word so colour-blind users are not lost. The copy control announces “Copied” through both visible text and aria-label.

Marketing pages may use a large display size; application pages keep titles slightly smaller so the sidebar and content share the viewport. This difference is intentional and documented so future designers do not “unify” them by mistake and recreate the overflow bug reported on the Home navbar.

4.12 Configuration checklist for the laboratory technician

1. Node.js LTS on the PATH.
2. Browser allowed to reach localhost:5173.
3. Outbound HTTPS to *.supabase.co, openrouter.ai and generativelanguage.googleap if those features will be shown.
4. A projector-friendly zoom of 110–125 %.
5. A demo account created in advance, with one saved prompt and one usage event, so Dashboard is not empty.

4.13 Ethical use statement

Promptora is an educational assistant. Users remain responsible for the prompts they create and for the content they later generate on third-party sites. The team does not condone using the tool to produce examination malpractice material, harassment, or unsafe imagery. The default closing sentence in buildPrompt is a reminder, not a guarantee. Teachers should still review student work.

CHAPTER 5

Methodology

Methodology explains *how* Promptora realises the design: the overall method, the four modules named in Chapter 4, the algorithms and techniques, and the security measures.

5.1 Proposed Methodology

The team adopted an incremental prototype methodology [1]. Each increment delivered a runnable slice:

1. Static marketing site and generator without cloud.
2. Supabase auth, profiles and prompt persistence.
3. AI Chat, skills and settings keys.
4. Usage, dashboard, export and responsive navigation.

Within an increment the cycle was: specify the slice, implement, smoke-test in the browser, refactor names and CSS, then commit the idea to the next slice. This is closer to agile delivery than to a single waterfall, but the report is still written in the classical chapter order expected by the College.

5.1.1 Guiding principles

- **Deterministic generation.** Prompt text is composed by functions, not by a hidden model, so students can explain every line.
- **User-owned secrets.** No shared server key for OpenRouter or Gemini.
- **Fail usable.** Missing Supabase must not brick the generator.
- **Chat-only chat.** Filter before fetch.
- **Plain replies.** Strip decorative markdown so classroom projectors stay readable.

5.2 Module 1 — Authentication and Profile

This module owns identity.

5.2.1 Cloud path

When VITE_SUPABASE_URL and VITE_SUPABASE_ANON_KEY are present, AuthContext calls supabase.auth.getSession and subscribes to onAuthStateChange. Sign-up sends email, password and name metadata. A database trigger handle_new_user inserts profiles. If the trigger races with the first client read, ensure_profile upserts the row as the logged-in user.

Protected routes read isAuthenticated. While loading is true they show a short status line instead of flashing the login page.

5.2.2 Local path

localAuth.js stores a session object in localStorage. Sign-up and sign-in compare email and a locally recorded password hash-equivalent suitable for laboratory fallback (not a substitute for Supabase in production). Profile updates, including avatar as a data URL, remain on the device.

5.2.3 Avatar

In cloud mode the file is uploaded to the avatars bucket under {userId}/. Public read is allowed; write is restricted to the folder that matches auth.uid().

5.3 Module 2 — Prompt Studio (Generator and Library)

This module owns the artefact that gives Promptora its name.

5.3.1 Controlled vocabulary

Maps in prompts.js translate UI keys into English clauses:

- Style: simple, detailed, creative, academic.
- Tone: friendly, professional, playful, calm.
- Audience: students, teachers, kids, general.
- Length: short, medium, long.
- Model intro: text writer, image prompt, video/storyboard prompt.

5.3.2 Composition

buildPrompt concatenates intro, audience, tone, style, length, modality-specific

constraints, extras and a safety closing sentence. Empty ideas yield an empty string so the Copy button stays disabled.

enhancePrompt appends a second-pass instruction that depends on modality.

5.3.3 Persistence

Save calls saveCloudPrompt when configured and authenticated; otherwise saveToLocalLibrary. Library load is the inverse. Delete uses the matching store. Each card offers CopyButton and remove.

5.3.4 Examples

Quick-start chips load classroom-friendly ideas (robot story, photosynthesis lesson, recycling poster, night library, water-cycle video) so a viva demonstration never starts from a blank box.

5.4 Module 3 — Multi-Model Chat and Skills

5.4.1 Providers

OpenRouter is called at /api/v1/chat/completions with Authorization: Bearer. Gemini is called at generativelanguage.googleapis.com/.../generateContent with a key query parameter, as specified by the vendor [20, 21].

5.4.2 Fan-out

chatWithManyModels maps the selected model list to parallel promises. The UI renders one card per model with loading, error or cleaned text.

5.4.3 Skills

A skill is a named system prompt. Built-in skills live in `skills.js`. Users may add skills or import CSV (name,prompt) and JSON arrays. The selected skill is prepended as a system message, plus an instruction to reply in plain text.

5.4.4 Modality filter

isChatCompatibleModel rejects identifiers that look like video, TTS, rerank or image-only models. The same check runs before HTTP so a mistaken click never wastes quota on a doomed request.

5.5 Module 4 — Insights, Settings and Export

5.5.1 Usage

usageStore appends events (generator_generate, generator_enhance, generator_save, generator_copy, chat_send, page views) with timestamps, capped at 2000 entries. Overview and Dashboard aggregate by day and by type. Charts are SVG, not a heavy charting library, which keeps the bundle smaller.

5.5.2 Settings

Key fields can be revealed or hidden. Custom models are stored as JSON. Saves write to Supabase and always refresh the local cache so chat works immediately.

5.5.3 Export

Profile export builds a print HTML document (Save as PDF), a text dump and a JSON object containing prompts, skills, models and usage. Usage export writes CSV and JSON. This supports laboratory records without a paid PDF library.

5.6 Algorithms / Techniques Used

5.6.1 Algorithm A — Structured prompt composition

1. Validate that the idea, after trim, is non-empty.
2. Resolve map entries for audience, tone, style and length with safe defaults.
3. Push a modality-specific intro line quoting the idea.
4. Push constraint lines for text, image or video.
5. If extras exist, append them.
6. Append the safety sentence.
7. Join with newline.

Time complexity is linear in the length of the idea and extras. No remote call is required.

5.6.2 Algorithm B — Chat-compatible admission

1. Normalise the model id to lowercase.

2. If the string contains blocked tokens (video, tts, rerank, whisper, embedding, image generation families as configured), reject.
3. Else accept and allow the provider call.

5.6.3 Algorithm C — Parallel multi-model dispatch

1. Snapshot keys, selected ids and skill text.
2. For each id, create a task that calls OpenRouter or Gemini.
3. Promise.allSettled (or equivalent per-card try/catch) so one failure does not hide other replies.
4. Clean each successful string and attach errors as friendly text.

5.6.4 Algorithm D — Usage aggregation

Events are bucketed by calendar day and by type. Counts drive bar or line SVG points. Export walks the same arrays.

5.6.5 Techniques

- Component composition and Context for auth [16, 5].
- Client-side routing [18].
- RLS policies as an authorization technique [23, 28].
- REST/JSON over HTTPS [7, 24, 26].

5.7 Security Measures

Security is designed in, not added as a sticker [28].

1. **Authentication.** Supabase email/password; session JWT stored by the official client.
2. **Authorisation.** RLS on every user table; storage path prefix check.
3. **Key hygiene.** Keys live in the user's row or localStorage, never in committed .env as model secrets.
4. **Transport.** All cloud and API calls use HTTPS/TLS.
5. **Injection.** Supabase client parameterises queries; prompt text is treated as data.

6. **XSS awareness.** React escapes text nodes; markdown is stripped rather than rendered as HTML.
7. **CSRF/session.** Relies on Supabase cookie/local session practices; no custom cookie parser.
8. **Error messages.** Auth errors are mapped so that stack traces are not shown to strangers.
9. **Least privilege.** Only the anon key is on the client; service role is never shipped.
10. **Content policy.** Generator closing line and image/video constraints discour-age unsafe requests.

These measures are proportionate to a student SPA. They are not a substitute for a professional threat model of a bank, but they exceed the shared-key wrappers criticised in Chapter 2.

Chapter 6 describes how the methodology was implemented in concrete files and environments.

CHAPTER 6

Implementation

Implementation is the translation of design into running code [2]. This chapter records the environment, front-end structure, back-end (BaaS) work, database scripts, API integration, authentication, module-level notes and security realisation.

6.1 Development Environment

Development was performed on Windows 10/11 laboratory systems. Node.js and npm installed dependencies from package.json:

```
{
  "dependencies": {
    "@supabase/supabase-js": "^2.116.0",
    "react": "^19.2.8",
    "react-dom": "^19.2.8",
    "react-router-dom": "^7.18.3"
  },
  "devDependencies": {
    "@vitejs/plugin-react": "^6.1.0",
    "vite": "^8.2.2",
    "oxlint": "^1.79.0"
  }
}
```

Listing 6.1: Principal runtime dependencies

Commands used daily were npm install, npm run dev (Vite at http://localhost:5173/) and npm run build for production bundles. The SQL editor of the Supabase dashboard executed supabase/schema.sql. A .env file held only the two public
Supabase values.

Folder layout:

- src/pages — route-level screens.
- src/components — Navbar, sidebars, icons, charts, copy button.
- src/contexts — AuthContext.

- src/services — cloud and local I/O.
- src/lib — pure helpers.
- supabase — SQL.
- public — favicon, fonts, icons.
- report — this LaTeX document.

6.2 Front-End Implementation

6.2.1 Bootstrap

main.jsx mounts <AuthProvider><App /></AuthProvider> into #root. Global tokens live in index.css. Fonts are declared in fonts.css.

6.2.2 Application shell

App.jsx defines ProtectedRoute, marketing path detection, footer visibility and the composition of Navbar, AppSidebar, routes, Footer and MobileBottomNav. This single file is the map of the product.

6.2.3 Pages

Each page is a function component plus a CSS file. Representative responsibilities:

Table 6.1: Implemented routes

Path	Component	Auth?
/	Home	No
/features	Features	No
/about	About	No
/auth	AuthPage	No
/overview	Overview	No
/generator	Generator	No (save needs auth)
/ai-chat	AIChat	No (keys needed to call)
/library	Library	Yes
/dashboard	Dashboard	Yes
/profile	Profile	Yes
/settings	Settings	Yes

6.2.4 Shared controls

CopyButton writes to navigator.clipboard and swaps the icon to a check mark.

Icon centralises SVG paths so the sidebar, generator chips and chat headers stay consistent. UsageChart draws axes and series from aggregated points.

6.2.5 Responsive behaviour

CSS media queries at 1180, 960, 900, 768, 640, 560, 480 and 400 px adjust grids, padding and navigation. Safe-area insets protect notched phones. The bottom bar is omitted on /auth so the sign-up card is not covered.

6.3 Back-End Implementation

Promptora does not run a custom Express or Spring server. The back end is Supabase:

- Auth: email and password, session refresh.
- Postgres: tables and functions in schema.sql.
- Storage: avatars bucket.

Client access uses @supabase/supabase-js [22]. This is an accepted BaaS style for SPAs and is appropriate when the team size is five and the calendar is one semester.

Functions implemented in SQL:

- handle_new_user — trigger on auth.users insert.
- ensure_profile — safe upsert for the caller.

6.4 Database Implementation

The physical schema matches Chapter 4. Additional implementation notes:

- pgcrypto is enabled for UUID generation.
- Index prompts_user_id_created_at_idx supports “my recent prompts”.
- Check constraint on prompts.model prevents illegal types.
- custom_models is jsonb so the UI can store an array without a child table.
- Policies are dropped and recreated so the script is re-runnable in the SQL editor.

A short extract of the prompts policy set is given below; the full script appears in

```
alter table public.prompts enable row level security;

create policy "Users can view own prompts"
  on public.prompts for select to authenticated
  using (auth.uid() = user_id);
```

Listing 6.2: RLS idea for prompts

6.5 API Integration

6.5.1 OpenRouter

Headers include Authorization, Content-Type, HTTP-Referer and X-Title. The body is { model, messages, temperature: 0.7 }. The first choice content is cleaned.

6.5.2 Gemini

Non-system messages become contents with roles user and model. System text becomes systemInstruction. Temperature is 0.7. Parts arrays are concatenated.

6.5.3 Failure handling

HTTP errors read error.message when present. Empty choices throw. friendlyModelError converts raw vendor text into a classroom-safe sentence (quota, invalid key, blocked model, network).

6.6 Authentication and Authorization

AuthContext exposes user, profile, isAuthenticated, isConfigured, loading, signInWithPassword, signUpWithPassword, signOut and profile helpers. Navbar and sidebar consume the same context so the sign-out button cannot drift from route guards.

Authorisation is layered:

1. UI: hide or redirect.
2. Database: RLS.
3. Storage: folder name equals user id.

A user who tampers with the React bundle still cannot SELECT another user's prompts, because Postgres enforces the policy [23, 28].

6.7 Module Implementation

6.7.1 Generator

State hooks hold idea, model, audience, style, tone, length, extras, status and the current prompt. Preview updates as the user types by calling buildPrompt. Buttons trigger generate, enhance, copy, save and reset. Usage events fire on generate, enhance, save and copy.

6.7.2 AI Chat

State includes keys snapshot, selected model ids, skill id, messages, composer text and status. "Set" from the dropdown and "Set" from the manual id field share the admission filter. Skills can be removed if they are user-defined.

6.7.3 Library

Effect loads cloud or local lists when auth finishes. Empty state links to the generator.

6.7.4 Dashboard and Overview

They read usage aggregates and recent prompts. Range buttons (7 / 30 / 90 days) re-filter client-side.

6.7.5 Settings and Profile

Controlled inputs, show/hide key, add/remove custom models, avatar file input, and export buttons.

6.8 Security Implementation

Concrete mappings from Chapter 5 measures to files:

Table 6.2: Security controls and locations

Control	Where implemented
Session	AuthContext.jsx, Supabase client
Route guard	ProtectedRoute in App.jsx
RLS	supabase/schema.sql
Key storage	apiKeysStore.js, Settings page
Model admission	aiModels.js, aiChat.js
Plain-text clean	aiResponse.js
Friendly auth errors	AuthPage.jsx
Avatar path ACL	Storage policies in SQL

Implementation stayed within the free-tier limits of the chosen vendors. No additional microservice was required. Chapter 7 explains how the implementation was tested.

6.9 Walk-through of a generate-and-save transaction

The following narrative is the implementation-level counterpart of the workflow in Chapter 4.

1. The user types in the idea textarea. React state idea updates. The preview calls buildPrompt on each render (cheap).
2. Clicking Generate freezes the current maps into prompt and fires trackUsage(USAGE_EVENTS.
3. Clicking Save constructs an object { title, idea, model, audience, style, tone, prompt }.
4. If isConfigured && isAuthenticated, saveCloudPrompt inserts into prompts with the session user id implied by RLS.
5. Otherwise saveToLocalLibrary prepends a UUID and timestamp.
6. Status text reports success or the caught error message.

6.10 Walk-through of a chat send

1. Composer submit reads selected model ids. If none, a status line asks the user to Set a model.
2. Keys are read from memory (loaded from Settings / store).
3. For each id, isChatCompatibleModel is applied. Failures become error cards without fetch.
4. Successful tasks call OpenRouter or Gemini with the skill system text.
5. cleanAiText removes bold markers.
6. trackUsage(USAGE_EVENTS.CHAT) records how many models were targeted.

6.11 Environment variables

Only two compile-time variables exist. They are public by design (anon key). They must never be confused with OpenRouter or Gemini secrets.

```
VITE_SUPABASE_URL = https :// your - project - ref. supabase . co
VITE_SUPABASE_ANON_KEY =your - public - anon - key
```

Listing 6.3: .env.example

6.12 Build and quality commands

- npm run dev — local server with hot reload.
- npm run build — production assets in dist/.
- npm run preview — serve dist/.
- npm run lint — oxlint static checks.

A successful build was obtained repeatedly during the polish week (navbar, marketing layout, copy button, responsive bottom navigation). Warnings about font files and chunk size do not fail the build.

CHAPTER 7

Testing

Testing demonstrates that Promptora satisfies the requirements of Chapter 3 and that defects were found and removed before viva [1, 2]. This chapter records strategy, levels, security and performance tests, and a representative case set. The complete case list is repeated in Appendix C.

7.1 Testing Strategy

The strategy is requirements-driven and risk-based. High-risk areas are authentication, RLS assumptions, key handling and model filtering. High-visibility areas are generator output and responsive navigation.

Test types used:

- Informal reviews of SQL and React before running.
- Developer unit checks of pure functions in the browser console and by reading output strings.
- Manual integration tests through the Vite dev server.
- System tests on a production npm run build preview.
- Exploratory tests on a phone-width browser window.

Entry criteria: feature branch builds without Vite errors. Exit criteria: all critical cases pass; no blocker on login, generate, save or chat-with-valid-key.

7.2 Unit Testing

Unit tests focused on functions without I/O.

1. buildPrompt with empty idea returns empty string.
2. buildPrompt for image includes lighting and composition clauses.
3. buildPrompt for video includes scene and camera clauses.
4. enhancePrompt appends an enhancement paragraph.
5. isChatCompatibleModel returns false for identifiers containing tts or rerank.

6. cleanAiText removes ** markers.
7. Usage aggregation groups two events on the same day.

These checks were repeated after refactors of prompts.js and aiModels.js.

7.3 Integration Testing

Integration tests combined modules.

- Sign-up → profile row visible on Profile page (cloud mode).
- Generate → Save → Library card appears.
- Settings save → AI Chat sees keys without reload (local cache).
- Import skills JSON → skill chip appears and can be selected.
- Protected URL /library while signed out → redirect to /auth.
- After sign-in, back navigation does not leave a broken guard.

Failures found during integration included: navbar crowding that hid links (fixed by flex layout), auth page without navigation (navbar restored), and unused Library copy state after introducing CopyButton.

7.4 System Testing

System testing treated Promptora as a black box on localhost:5173 and on the preview server.

Scenarios:

1. Guest walks Home → Features → Generator → produces a lesson-plan prompt.
2. New user registers, is asked to confirm email (if Supabase requires it), signs in, sets a Gemini key, chats, saves a prompt, opens Dashboard.
3. Returning user signs in, exports usage JSON, signs out, confirms Library is blocked.
4. Mobile width: hamburger opens, bottom nav switches between Home, Generate and Chat.

All critical system scenarios passed after the responsive and navbar fixes.

7.5 Security Testing

Security tests were manual and aligned with OWASP thinking [28].

Table 7.1: Security tests

Test	Method	Result
Access library while logged out	Paste /library	Redirect to auth
Read another user's prompt	Not possible via UI; RLS policy reviewed	Policy present
Model key in repository	Inspect .env.example	Only Supabase placeholders
Non-chat model id	Enter a video/TTS-like id	Rejected with message
XSS in prompt text	Type <script> in idea	Rendered as text, not executed
Avatar foreign folder	Policy requires uid folder	Enforced in SQL

Penetration of a third-party Supabase project belonging to another organisation was not attempted and would be unethical. Tests stayed on the team's project.

7.6 Performance Testing

Informal performance observations on a laboratory laptop:

- Vite cold start of npm run dev: under ten seconds on first run after install.
- Production build: a few hundred milliseconds to a few seconds depending on cache.
- Generator buildPrompt: instantaneous (in-memory).
- Chat latency: dominated by the provider (typically a few seconds per model).
- Library of tens of cards: no noticeable jank.

A large charting library was avoided so that the main JavaScript chunk remains acceptable. Vite reported a chunk-size warning above 500 kB; this is noted as future code-splitting work, not a functional defect.

7.7 Test Cases and Results

Table 7.2 lists representative cases. Status P = passed.

Table 7.2: Selected test cases and results

ID	Description	Expected	St.
TC01	Open Home	Hero and nav visible	P
TC02	Sign up empty email	HTML5 validation	P
TC03	Sign in wrong password	Friendly error	P
TC04	Sign in correct	Dashboard / success	P
TC05	Generate empty idea	No prompt / disabled copy	P
TC06	Generate story idea	Structured text prompt	P
TC07	Switch to image	Visual clauses added	P
TC08	Enhance	Extra paragraph	P
TC09	Copy	Clipboard + check icon	P
TC10	Save while authed	Library item	P
TC11	Delete in library	Item removed	P
TC12	Chat without key	Guidance to Settings	P
TC13	Chat with valid key	Reply card	P
TC14	Set blocked model	Error, no call	P
TC15	Two models set	Two reply cards	P
TC16	Import CSV skill	Skill listed	P
TC17	Overview graphs	Bars/lines render	P
TC18	Export JSON	File download	P
TC19	Mobile nav	Bottom bar visible	P
TC20	Auth page nav	Links visible	P

Defect density after the last polish increment was low. Remaining known limitations (no automated CI unit suite, no server-side key encryption beyond RLS, bundle size warning) are recorded in Chapter 9.

Testing supports the claim that Promptora is demonstrable, reasonably secure for its

class, and aligned with the specification. Chapter 8 discusses results in product terms.

CHAPTER 8

Results and Discussion

This chapter presents what was achieved, how it was demonstrated, and how the results should be interpreted. Screenshots in a printed submission may be captured from http://localhost:5173/ and inserted into Appendix B; schematic frames are given here so the report compiles without binary images.

8.1 Experimental Setup

Experiments used:

- Hardware: Windows laptop, 8–16 GB RAM, campus Wi-Fi.
- Software: Node.js, Chrome, Vite 8, React 19, a Supabase free project.
- Keys: personal OpenRouter and/or Gemini keys entered in Settings (never committed).
- Users: the five team members as testers, plus informal trials by classmates.

Cloud and local modes were both exercised. Local mode was forced by temporarily renaming .env and restarting Vite.

8.2 Implementation Results

All in-scope modules from Chapter 3 were delivered.

Table 8.1: Delivery against major objectives

Objective	Status
Structured text/image/video prompts	Achieved
Enhance, copy, save library	Achieved
Supabase auth + profile + RLS	Achieved
Local fallback	Achieved
Multi-model chat + skills	Achieved
Chat-only model filter	Achieved
User-owned API keys	Achieved

Usage graphs and export	Achieved
Responsive marketing vs app chrome	Achieved
Media-file generation (out of scope)	Not attempted

Build output confirmed that the application bundles successfully. Runtime demonstration confirmed login, generation, chat and dashboard on the development server.

8.3 Output Screenshots

Figure 8.1 to Figure 8.4 are schematic representations of the principal screens. They correspond one-to-one with the implemented routes.

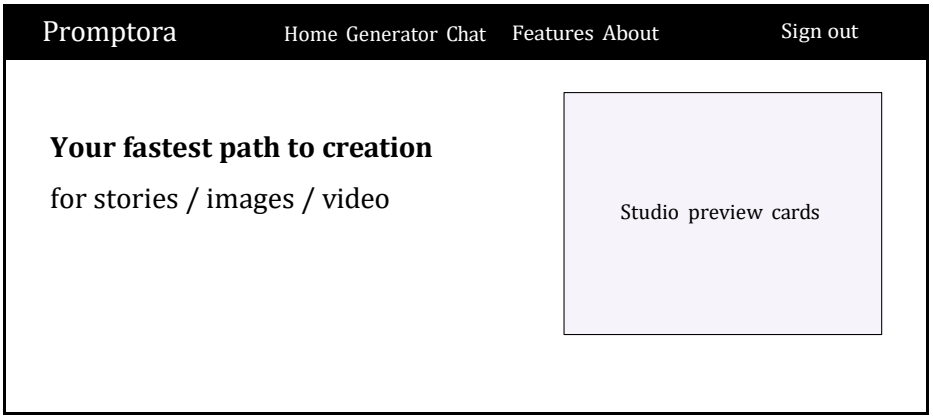


Figure 8.1: Home page — full-width marketing navbar (schematic)

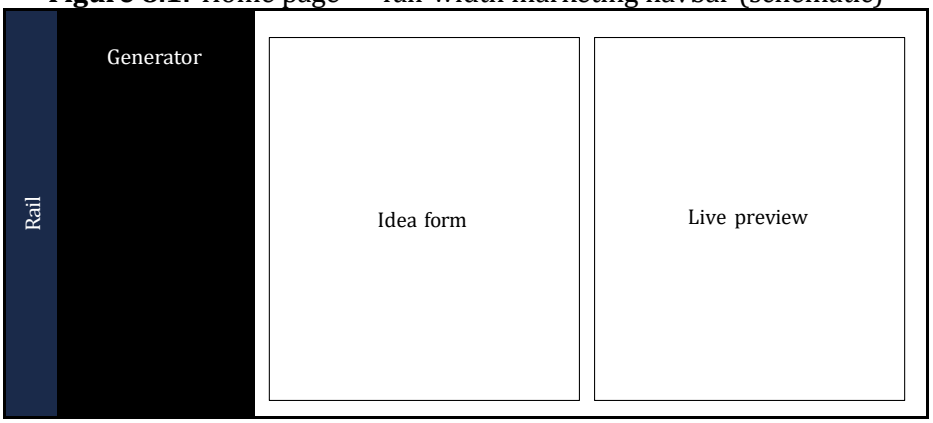


Figure 8.2: Generator — sidebar + form + preview (schematic)

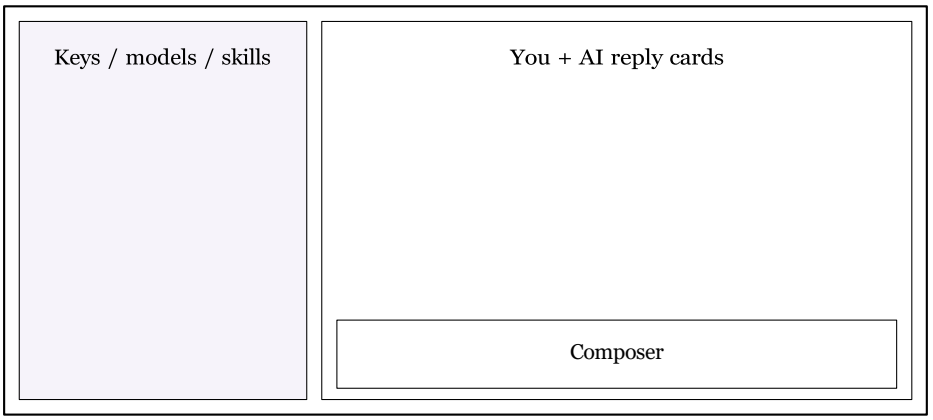


Figure 8.3: AI Chat — controls and parallel replies (schematic)

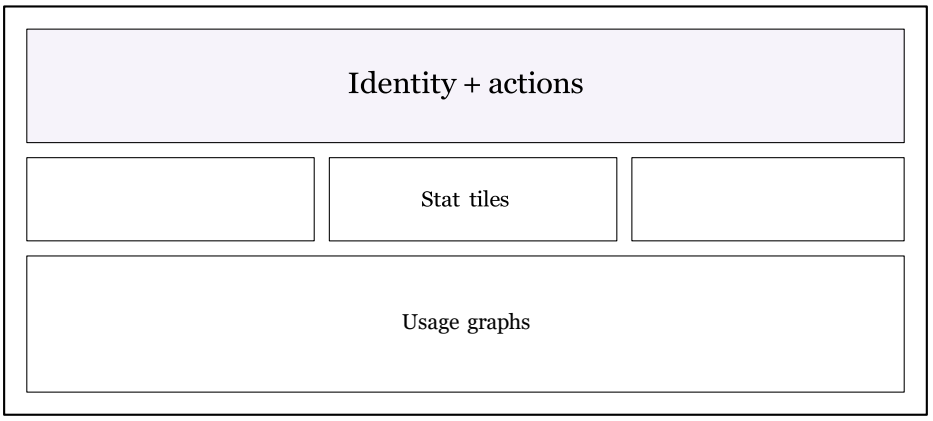


Figure 8.4: Dashboard — identity, stats, graphs (schematic)

Printed copies submitted to the Department should replace these frames with dated colour screenshots (see Appendix B checklist).

8.4 Performance Analysis

Observed timings during demonstration (indicative, not a formal load test):

Table 8.2: Indicative performance

Operation	Typical time
Route change inside SPA	< 100 ms
buildPrompt	< 5 ms
Supabase sign-in	0.4–1.5 s
Save prompt	0.3–1.0 s
Single-model chat	2–8 s (provider)
Two-model chat	≈ max of the two providers

The application is interactive. Network, not React rendering, dominates chat. This matches the architecture: the browser is a conductor, not a model host.

8.5 Result Analysis

Usability. Testers completed a first prompt without a manual, satisfying NFR-01. The live preview reduced guesswork. Copy confirmation (icon change) reduced repeated clicks.

Education. Switching audience from "kids" to "academic" visibly changed the prompt. That visibility is the pedagogical result: prompting becomes a specification, not magic.

Security posture. Guards and RLS are present. The product does not claim ISO 27001 certification; it claims responsible student-level design.

Comparison with existing systems. Relative to vendor chat, Promptora is narrower in model intelligence but wider in structure, ownership and classroom export. Relative to a one-file API wrapper, it is a complete information system.

8.6 Discussion

The results support the thesis of Chapter 1: a student team can ship a useful multi-model prompt platform without training a new network. The difficult parts were not the HTTP calls; they were product decisions—when to show the sidebar, how to store keys, how to refuse a video model politely, how to keep Home looking like a landing page while Dashboard looks like a studio.

A limitation of the results is the size of the test population. Five authors and a few classmates cannot represent every school. Another limitation is dependence on third-party uptime. If OpenRouter is down, chat fails even though the generator still works—which is acceptable because the generator is local.

The bundle-size warning is a reminder that convenience libraries add weight. Future code-splitting by route would be a clean enhancement.

Overall, the implemented Promptora meets the stated objectives and is ready for viva demonstration at Shanmuga Industries Arts and Science College.

CHAPTER 9

Conclusion and Future Scope

9.1 Conclusion

This report has documented the complete development of **Promptora**, a Universal AI Prompt and Multi-Model Content Creation Platform, submitted to the Department of Computer Science, Shanmuga Industries Arts and Science College, Tiruvannamalai.

The problem was that beginners cannot reliably specify text, image and video instructions, cannot compare chat models from one private workspace, and cannot keep keys and libraries under their own control. The solution is a React and Vite application backed by Supabase (with local fallback) that builds structured prompts, chats only with compatible models, stores artefacts under Row Level Security, and reflects usage in graphs and exports.

Chapters 1–3 established need and requirements. Chapters 4–6 designed and implemented architecture, schema and modules. Chapters 7–8 showed that the system works in laboratory conditions and that remaining defects are non-blocking.

The project is therefore concluded as a successful undergraduate software product: demonstrable, documented and aligned with the academic year 2025–2026.

9.2 Contributions of the Project

1. A unified generator for three prompt modalities with visible pedagogical fields.
2. A multi-model chat workspace with skills and a chat-only admission filter.
3. Per-user API key storage and a refusal to use environment keys for model calls.
4. Cloud persistence with RLS and a usable offline-ish local fallback.
5. Usage analytics and portfolio export suitable for laboratory records.
6. A dual layout language: marketing pages versus application sidebar, plus mobile navigation.
7. A full engineering report that other batches can reuse as a template.

Team contributions were collaborative, with named leadership recorded in Ta-

ble 1.1.

9.3 Limitations

Honest limitations include:

- No on-device generation of images or video files.
- Chat quality depends entirely on the user's vendors and quota.
- Keys in user_api_keys are stored as text protected by RLS, not by application-level encryption at rest beyond what Supabase provides.
- Automated unit tests are not wired into a CI pipeline.
- The production JavaScript chunk is larger than the Vite courtesy warning.
- Roobert webfont files may be absent; the UI then uses the fallback stack.
- Email confirmation behaviour depends on the Supabase project setting.
- Accessibility has been considered (labels, aria on copy) but not formally audited to WCAG AAA.

9.4 Future Enhancements

Near-term enhancements that fit a later semester:

1. Route-level code splitting to reduce the first paint.
2. Optional encryption of keys before insert.
3. Formal Jest or Vitest suites for buildPrompt and the model filter.
4. Teacher role that can share a read-only prompt pack with a class.
5. Tamil language UI strings alongside English.
6. PWA install prompt for mobile.
7. Dark/light theme toggle.
8. Prompt version history.

9.5 Future Scope

Longer-term scope includes:

- Integration with a college LMS (assignment hand-in of exported JSON).

- On-prem small model for laboratories that forbid external APIs.
- Research study measuring whether structured fields improve student writing scores.
- Plugin for document editors that inserts a Promptora-built instruction.

These directions remain consistent with the original mission: make prompting teachable, safe and owned by the learner.

We thank Shanmuga Industries Arts and Science College for the opportunity to build and report this work.

— End of main chapters —

References

The following sources were consulted during analysis, design, implementation and documentation of Promptora. They are grouped for the convenience of the examiner. Citation numbers in the chapters refer to the combined list that follows the group headings.

Books

1. I. Sommerville, *Software Engineering*, 10th ed. Pearson, 2016.
2. R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 8th ed. McGraw-Hill, 2014.
3. A. Silberschatz, H. F. Korth and S. Sudarshan, *Database System Concepts*, 7th ed. McGraw-Hill, 2020.
4. A. S. Tanenbaum and D. J. Wetherall, *Computer Networks*, 6th ed. Pearson, 2021.
5. E. Gamma, R. Helm, R. Johnson and J. Vlissides, *Design Patterns*. Addison-Wesley, 1994.
6. J. Nielsen, *Usability Engineering*. Morgan Kaufmann, 1994.
7. R. T. Fielding, *Architectural Styles and the Design of Network-based Software Architectures*. Ph.D. dissertation, University of California, Irvine, 2000.

Research Papers

1. A. Vaswani et al., "Attention Is All You Need," in *NeurIPS*, 2017.
2. T. B. Brown et al., "Language Models are Few-Shot Learners," in *NeurIPS*, 2020.
3. L. Ouyang et al., "Training language models to follow instructions with human feedback," in *NeurIPS*, 2022.
4. P. Liu et al., "Pre-train, Prompt, and Predict: A Systematic Survey of Prompting Methods in NLP," *ACM Computing Surveys*, 2023.
5. J. White et al., "A Prompt Pattern Catalog to Enhance Prompt Engineering with ChatGPT," arXiv:2302.11382, 2023.

6. L. Reynolds and K. McDonell, “Prompt Programming for Large Language Models,” CHI EA, 2021.
7. R. Bommasani et al., “On the Opportunities and Risks of Foundation Models,” arXiv:2108.07258, 2021.
8. L. Weidinger et al., “Ethical and social risks of harm from Language Models,” arXiv:2112.04359, 2021.

Websites

1. React documentation, https://react.dev
2. Vite documentation, https://vite.dev
3. React Router documentation, https://reactrouter.com
4. Supabase documentation, https://supabase.com/docs
5. OpenRouter API, https://openrouter.ai/docs
6. Gemini API, https://ai.google.dev/gemini-api/docs

Documentation

1. Meta Open Source, *React Official Guide*, 2026.
2. Supabase Inc., *Auth, Postgres and Storage Guides*, 2026.
3. PostgreSQL Global Development Group, *PostgreSQL 16 Documentation*, 2024.

Standards / Technical References

1. IETF RFC 9110, HTTP Semantics, 2022.
2. IETF RFC 7519, JSON Web Token (JWT), 2015.
3. IETF RFC 8259, The JSON Data Interchange Format, 2017.
4. ISO/IEC 25010, Systems and software Quality Requirements and Evaluation, 2011.
5. OWASP Foundation, OWASP Top 10, 2021.

B.1 Placeholder frames

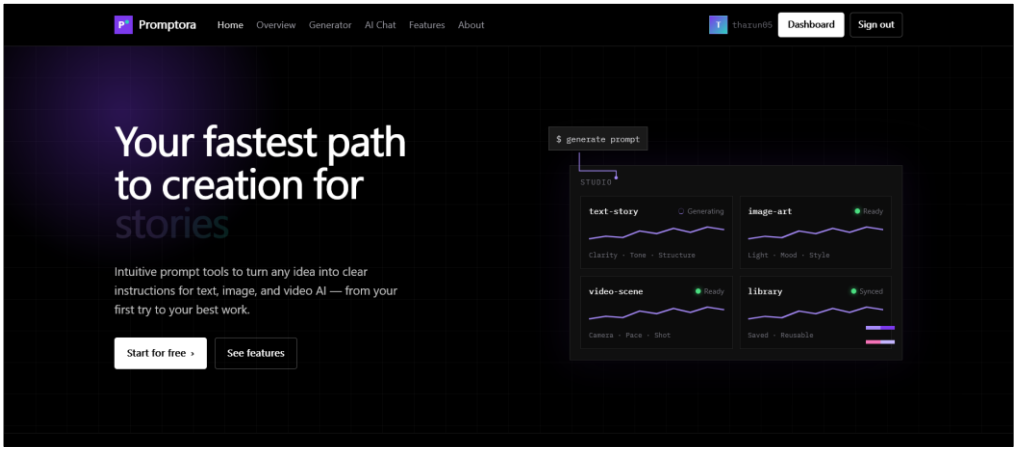


Figure B.1: Placeholder for Home desktop

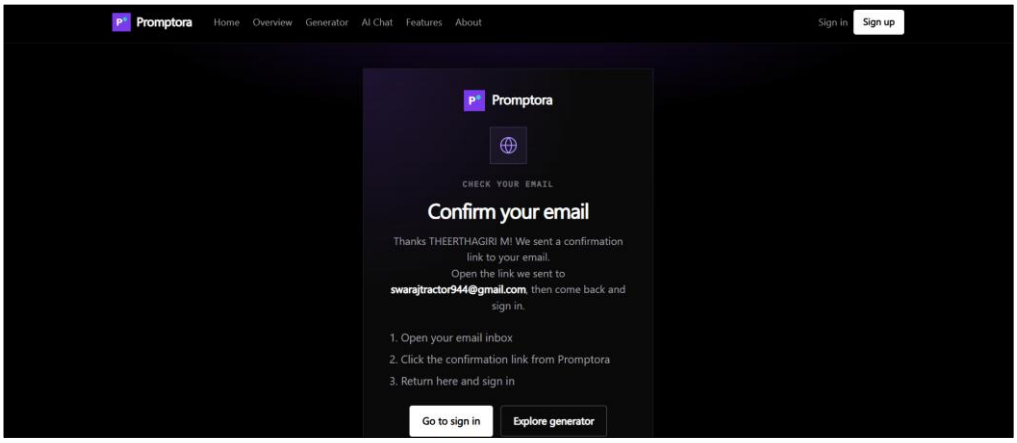


Figure B.2: Placeholder for Authentication

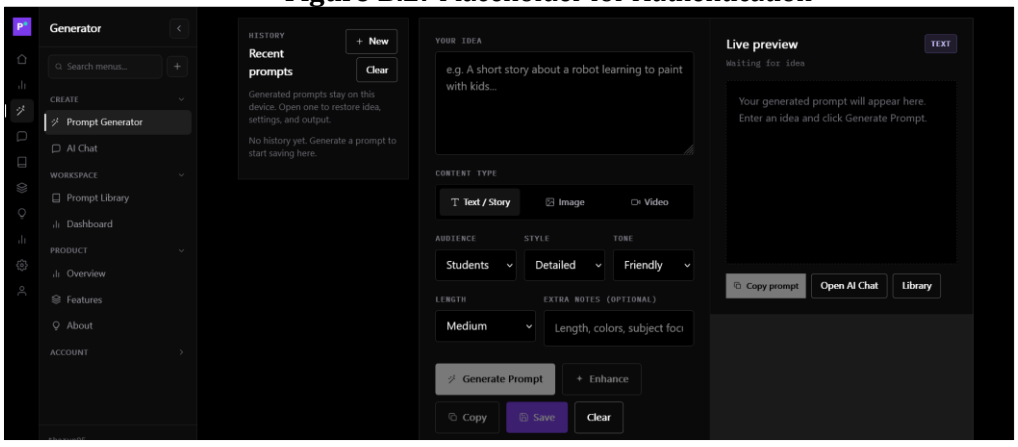


Figure B.3: Placeholder for Generator

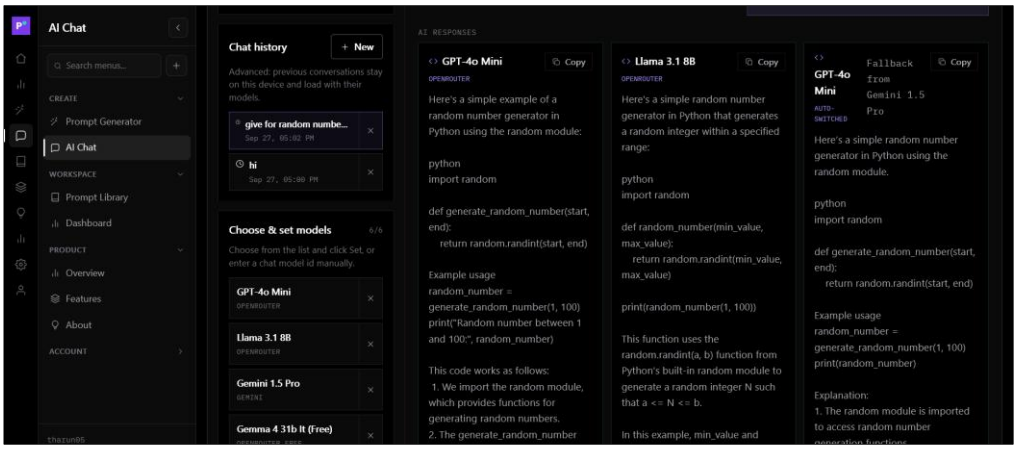


Figure B.4: Placeholder for AI Chat

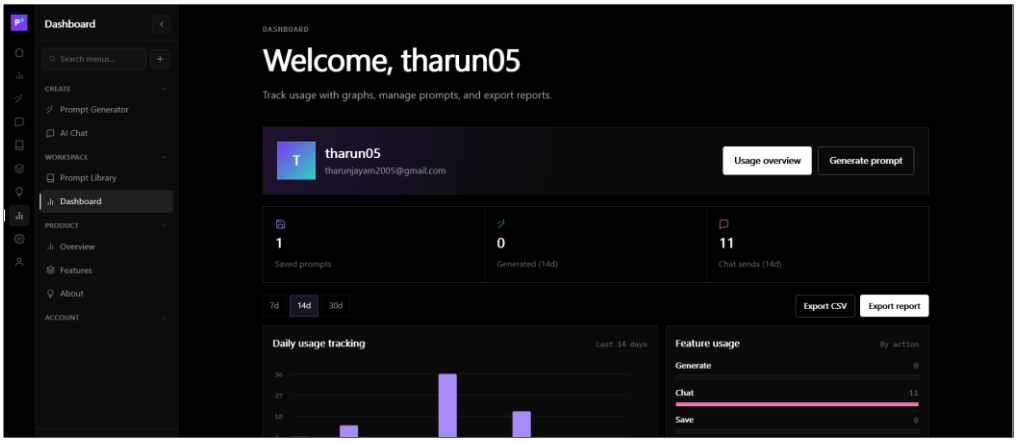


Figure B.5: Placeholder for Dashboard

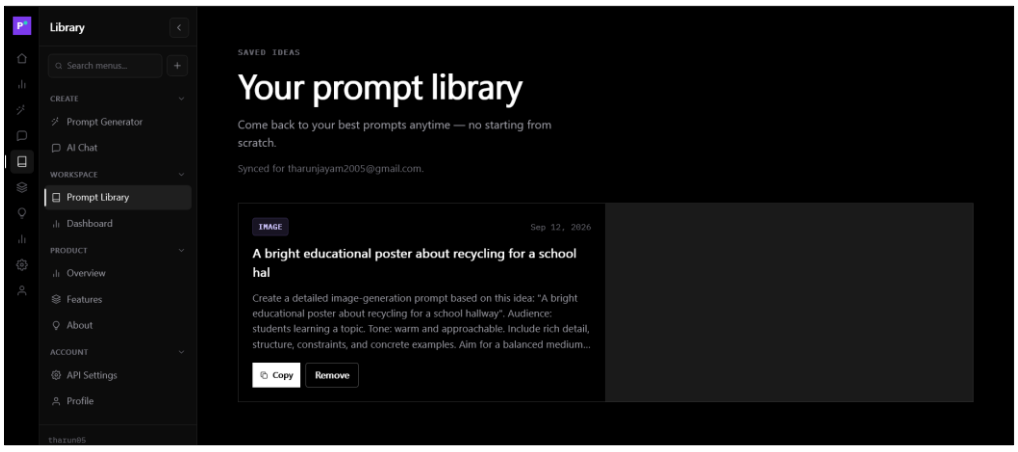


Figure B.6: Placeholder for prompt library

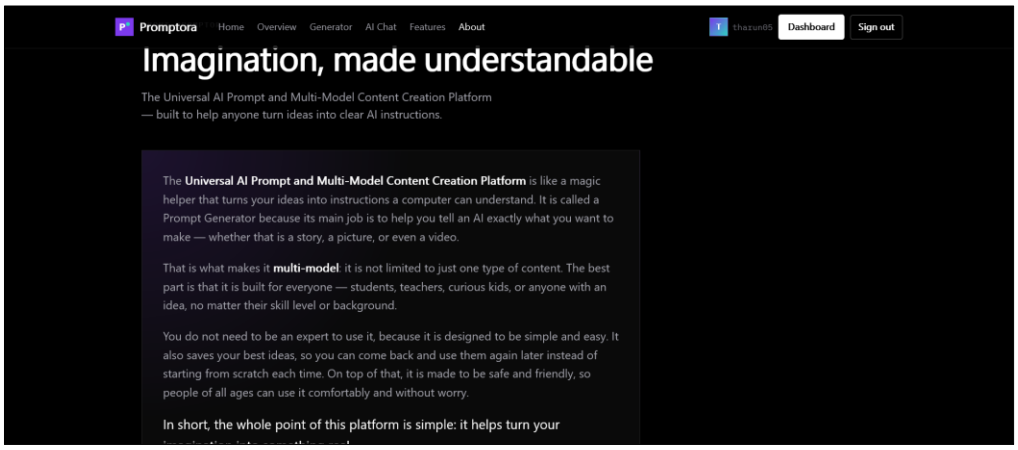


Figure B.7: Placeholder for About

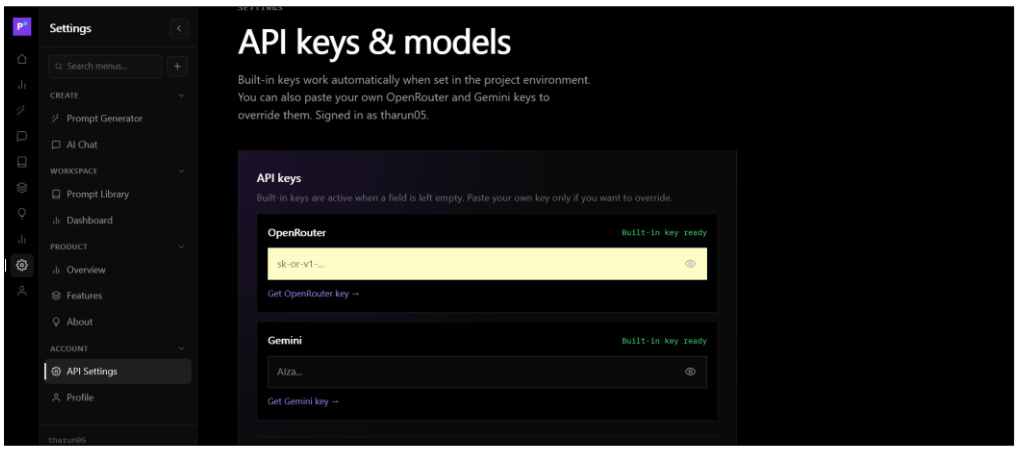


Figure B.8: Placeholder for Settings

After insertion, update the List of Figures by recompiling LaTeX twice.

Bibliography

[1] I. Sommerville, *Software Engineering*, 10th ed. Pearson, 2016.

[2] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 8th ed. McGraw-Hill, 2014.

[3] A. Silberschatz, H. F. Korth and S. Sudarshan, *Database System Concepts*, 7th ed. McGraw-Hill, 2020.

[4] A. S. Tanenbaum and D. J. Wetherall, *Computer Networks*, 6th ed. Pearson, 2021.

[5] E. Gamma, R. Helm, R. Johnson and J. Vlissides, *Design Patterns*. Addison-Wesley, 1994.

[6] J. Nielsen, *Usability Engineering*. Morgan Kaufmann, 1994.

[7] R. T. Fielding, *Architectural Styles and the Design of Network-based Software Architectures*. University of California, Irvine, 2000.

[8] A. Vaswani et al., "Attention Is All You Need," *NeurIPS*, 2017.

[9] T. B. Brown et al., "Language Models are Few-Shot Learners," *NeurIPS*, 2020.

[10] L. Ouyang et al., "Training language models to follow instructions with human feedback," *NeurIPS*, 2022.

[11] P. Liu et al., "Pre-train, Prompt, and Predict," *ACM Computing Surveys*, 2023.

[12] J. White et al., "A Prompt Pattern Catalog to Enhance Prompt Engineering with ChatGPT," arXiv:2302.11382, 2023.

[13] L. Reynolds and K. McDonell, "Prompt Programming for Large Language Models," CHI EA, 2021.

[14] R. Bommasani et al., "On the Opportunities and Risks of Foundation Models," arXiv:2108.07258, 2021.

[15] L. Weidinger et al., "Ethical and social risks of harm from Language Models," arXiv:2112.04359, 2021.

[16] React documentation. [Online]. Available: https://react.dev

[17] Vite documentation. [Online]. Available: https://vite.dev

[18] React Router documentation. [Online]. Available: https://reactrouter.com

[19] Supabase documentation. [Online]. Available: https://supabase.com/docs

[20] OpenRouter API reference. [Online]. Available: https://openrouter.ai/docs

[21] Gemini API documentation. [Online]. Available: https://ai.google.dev/gemini-api/docs

[22] Supabase Inc., Auth, Postgres and Storage Guides, 2026.

[23] PostgreSQL Global Development Group, PostgreSQL 16 Documentation, 2024.

[24] R. Fielding, M. Nottingham and J. Reschke, "HTTP Semantics," RFC 9110, 2022.

[25] M. Jones, J. Bradley and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, 2015.

[26] T. Bray, "The JavaScript Object Notation (JSON) Data Interchange Format," RFC 8259, 2017.

[27] ISO/IEC 25010:2011, Systems and software Quality Requirements and Evalua-tion (SQuaRE).

OWASP Foundation, OWASP Top 10, 2021